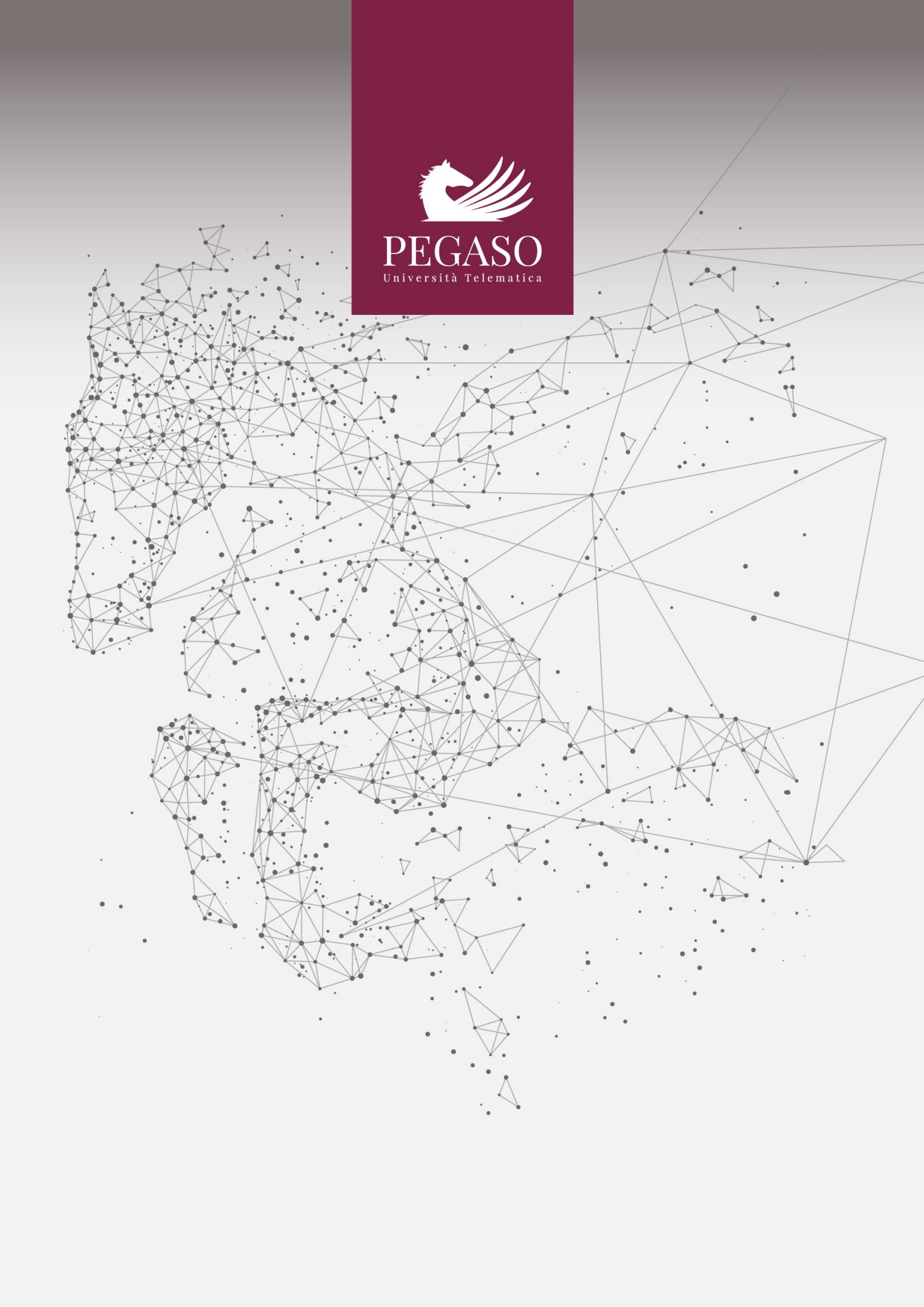

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. PROGRAMMAZIONE DINAMICA 3
2. IMPLEMENTAZIONE 5
3. MEMOIZATION 8
4. DIVIDE ET IMPERA VS PROGRAMMAZIONE DINAMICA 10
BIBLIOGRAFIA 12

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Programmazione dinamica

Quando dobbiamo affrontare un problema possiamo approcciare diverse tecniche di risoluzione:

- Divide-et-impera
- Programmazione dinamica / memoization
- Tecnica greedy
- Ricerca locale
- Backtrack
- Algoritmi probabilistici
- Tecniche di soluzione per problemi intrattabili

Focalizziamo la nostra attenzione sulla **programmazione dinamica**.

La tecnica di programmazione dinamica, simile al metodo divide et impera, permette di risolvere problemi combinando le soluzioni dei sottoproblemi. Tuttavia, mentre il metodo divide et impera suddivide il problema in sottoproblemi indipendenti, la programmazione dinamica si applica quando i sottoproblemi hanno in comune dei sottosottoproblemi. In questo modo, un algoritmo divide et impera può svolgere più lavoro del necessario, risolvendo ripetutamente i sottosottoproblemi comuni.

Per evitare questo problema, la programmazione dinamica risolve ogni sottoproblema una sola volta e salva la sua soluzione in una tabella. Ciò permette di evitare il lavoro di ricalcolare la soluzione ogni volta che si presenta il sottoproblema. La programmazione dinamica si applica in particolare ai problemi di ottimizzazione, dove esistono molte soluzioni possibili, ciascuna con un valore associato. L'obiettivo è trovare la soluzione ottima, ovvero quella con il valore massimo o minimo.

Il processo di sviluppo di un algoritmo di programmazione dinamica può essere suddiviso in quattro fasi:

1. nella prima fase si cerca di caratterizzare la struttura di una soluzione ottima
2. nella seconda fase si definisce in modo ricorsivo il valore di una soluzione ottima
3. nella terza fase si calcola il valore di una soluzione ottima secondo uno schema bottom-up, ovvero partendo dal basso verso l'alto
4. nella quarta fase si costruisce la soluzione ottima dalle informazioni calcolate

Le prime tre fasi formano la base per risolvere un problema applicando la programmazione dinamica, mentre la quarta fase può essere omessa se si cerca soltanto il valore di una soluzione ottima.

Tuttavia, a volte durante la fase 4 si inseriscono informazioni aggiuntive per ottimizzare ulteriormente la soluzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sintetizzando:

- Si divide un problema ricorsivamente in sottoproblemi
- Ogni sottoproblema viene risolto una volta sola
- La sua soluzione viene memorizzata in una tabella
- Nel caso un sottoproblema debba essere risolto **nuovamente**, si ricerca la sua soluzione dalla tabella (essendo già stato risolto, la soluzione è stata scritta in tabella).

ATTENZIONE: la tabella è facilmente indirizzabile – lookup in $O(1)$

L'approccio generale è quello per cui abbiamo a che fare con problemi di ottimizzazione per i quali cerchiamo la definizione della soluzione in maniera ricorsiva, in particolare il valore della soluzione in maniera ricorsiva (essendo un problema di ottimizzazione, il valore sarà ad es. un minimo, un massimo ecc.); a questo punto, se vi sono dei problemi "ripetuti" posso utilizzare la **programmazione dinamica** o una sua specializzazione che è la **memoization**. Ovviamente, se non vi sono problemi ripetuti posso utilizzare un classico approccio **divide et impera**.

Sia la programmazione dinamica che la memoization producono una tabella delle soluzioni.

Ricordiamo che il termine programmazione dinamica (Dynamic Programming) è stato coniato da Richard Bellman agli inizi degli anni '50, nell'ambito dell'ottimizzazione matematica: inizialmente, si riferiva al processo di risolvere un problema compiendo le migliori decisioni una dopo l'altra:

- "Dynamic" doveva dare un senso "temporale"
- "Programming" si riferiva all'idea di creare "programmazioni ottime", per esempio nel campo della logistica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Implementazione

Ecco un esempio di codice in Python che utilizza la programmazione dinamica per calcolare il numero di modi in cui è possibile raggiungere una determinata posizione in una griglia rettangolare, partendo dall'angolo in alto a sinistra.

Supponiamo che la griglia abbia dimensioni $m \ x \ n$, dove m è il numero di righe e n è il numero di colonne. La posizione iniziale è l'angolo in alto a sinistra della griglia, cioè la posizione $(0,0)$. La posizione finale è l'angolo in basso a destra della griglia, cioè la posizione $(m-1, n-1)$.

Il codice utilizza una matrice DP di dimensioni $m \ x \ n$ per memorizzare i risultati intermedi. L'elemento $DP[i][j]$ contiene il numero di modi in cui è possibile raggiungere la posizione (i,j).

```
def num_ways(m, n):
    # Initialize the dynamic programming matrix with zeros
    DP = [[0] * n for _ in range(m)]

    # There's only one way to reach the top row - the left column
    for i in range(m):
        DP [i][0] = 1
    for j in range(n):
        DP [0][j] = 1

    # Fill in the rest of the matrix using dynamic programming
    for i in range(1, m):
        for j in range(1, n):
            DP[i][j] = DP[i-1][j] + DP[i][j-1]

    # Return the number of ways to reach the bottom right corner
    return DP[m-1][n-1]
```

Per esempio, per calcolare il numero di modi in cui è possibile raggiungere la posizione $(2,2)$ in una griglia rettangolare di dimensioni $3 \ x \ 3$, si può chiamare la funzione in questo modo:

```
num_ways(3, 3)  # Output: 6
```

Il risultato sarà 6, perché ci sono sei modi diversi per raggiungere la posizione (2, 2) partendo dall'angolo in alto a sinistra della griglia.

Consideriamo un altro problema: **Maximum Subarray Problem** (problema della massima sottosequenza). Il problema consiste nel trovare la sottosequenza di numeri contigui in un array di numeri che ha la somma massima.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ad esempio, consideriamo l'array $[2,-5,3,1,-2,4,-1]$. La sottosequenza $[3,1,-2,4]$ ha somma massima pari a 6, che è il risultato che ci aspettiamo dalla funzione max_subarray_sum.

Il problema può essere risolto utilizzando la tecnica della programmazione dinamica: l'idea è di utilizzare una matrice DP di dimensioni $n\ x\ 2$, dove n è la lunghezza dell'array e $DP[i][0]$ contiene la somma massima ottenibile utilizzando i primi i numeri dell'array senza includere l'$i-esimo$ numero, mentre $DP[i][1]$ contiene la somma massima ottenibile utilizzando i primi i numeri dell'array includendo l'i-esimo numero.

Ecco il codice:

```
def max_subarray_sum(arr):
    n = len(arr)
    dp = [[0] * 2 for _ in range(n)]

    # Initialize the DP matrix with the first element of the array
    dp[0][0] = 0
    dp[0][1] = arr[0]

    # Fill in the rest of the matrix using dynamic programming
    for i in range(1, n):
        dp[i][0] = max(dp[i-1][0], dp[i-1][1])
        dp[i][1] = max(dp[i-1][1] + arr[i], arr[i])

    # Return the maximum sum
    return max(dp[n-1][0], dp[n-1][1])
```

Per esempio, per trovare la somma massima di una sottosequenza di numeri nell'array $[2,-5,3,1,-2,4,-1]$, si può chiamare la funzione in questo modo:

```
max_subarray_sum([2, -5, 3, 1, -2, 4, -1])  # Output: 7
```

Il risultato sarà 7, perché la sottosequenza $[3,1,-2,4]$ ha somma massima pari a 7.

Introduciamo ora il problema della distanza di Levenshtein tra due stringhe.

Il problema consiste nel trovare il numero minimo di operazioni di inserimento, cancellazione e sostituzione di un singolo carattere necessarie per trasformare una stringa in un'altra.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito il codice:

```
def levenshtein_distance(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n+1) for _ in range(m+1)]

    # Initialize the DP matrix with the base cases
    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j

    # Fill in the rest of the matrix using dynamic programming
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s[i-1] == t[j-1]:
                dp[i][j]=dp[i-1][j-1]
            else:
                dp[i][j]=1+min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1])

    # Return the Levenshtein distance
    return dp[m][n]
```

Per esempio, per calcolare la distanza di Levenshtein tra le stringhe "casa" e "pasta", si può chiamare la funzione in questo modo:

```
levenshtein_distance("casa", "pasta")  # Output: 2
```

Il risultato sarà 2, perché sono necessarie due operazioni di sostituzione ("c" diventa "p" e "s" diventa "t") per trasformare la stringa "casa" nella stringa "pasta".

La funzione levenshtein_distance utilizza la programmazione dinamica per riempire una matrice DP di dimensioni $m+1\ x\ n+1$, dove m e n sono le lunghezze delle due stringhe, con la distanza di Levenshtein tra le sottosequenze di lunghezza i e j della prima e della seconda stringa, rispettivamente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Memoization

Consideriamo il calcolo di Fibonacci nelle seguenti modalità:

```
def fibonacci_rec(n):
    if n < 2:
        return n
    else:
        return fibonacci_rec(n-1) + fibonacci_rec(n-2)

def fibonacci_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n < 2:
        result = n
    else:
        result=fibonacci_memo(n-1,memo)+fibonacci_memo(n-2,memo)
    memo[n] = result
    return result

def fibonacci_dp(n):
    if n < 2:
        return n
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]
```

Abbiamo 3 modalità differenti di calcolo di Fibonacci:

- **fibonacci_rec**: in questa funzione, utilizziamo la ricorsione per calcolare il numero di Fibonacci per un dato indice n; se $n < 2$, restituiamo direttamente il valore di n, altrimenti calcoliamo il numero di Fibonacci per $n - 1$ e $n - 2$ e li sommiamo
- **fibonacci_dp**: in questa funzione, utilizziamo la programmazione dinamica per memorizzare i valori intermedi e ridurre la complessità computazionale del calcolo dei numeri di Fibonacci:
    - inizializziamo una lista dp con i valori 0 e 1, che rappresentano i primi due numeri di Fibonacci
    - utilizziamo un ciclo for per calcolare i numeri di Fibonacci da 2 a n, utilizzando i valori precedenti memorizzati nella lista dp
    - infine, restituiamo il valore $dp[n]$, che rappresenta il numero di Fibonacci richiesto

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **fibonacci_memo**: in questa funzione, utilizziamo la **memoization** per evitare di ricalcolare i valori intermedi

Analizziamo cosa significa **memoization**.

La funzione riceve due argomenti: n è l'indice del numero di Fibonacci che vogliamo calcolare, mentre $memo$ è un dizionario che utilizzeremo per memorizzare i valori intermedi. Se il valore di n è già presente in memo, restituiamo il valore memorizzato anziché calcolarlo nuovamente; se $n < 2$, restituiamo direttamente il valore di n, altrimenti calcoliamo il numero di Fibonacci per $n - 1$ e $n - 2$ utilizzando la funzione **fibonacci_memo** in modo ricorsivo e li sommiamo; infine, memorizziamo il valore di result nel dizionario $memo$ e lo restituiamo.

Il metodo più vantaggioso in termini di complessità computazionale per il calcolo del numero di Fibonacci è la programmazione dinamica.

Analizziamo la complessità.

La complessità computazionale del calcolo del numero di Fibonacci con la ricorsione è esponenziale, ovvero cresce in modo molto rapido con l'aumentare di n, questo perché la funzione ricorsiva deve calcolare nuovamente i numeri di Fibonacci precedenti ogni volta che viene chiamata per un nuovo valore di n.

La memoization riduce la complessità computazionale del calcolo del numero di Fibonacci rispetto alla ricorsione, perché consente di evitare di ricalcolare i valori intermedi già calcolati in precedenza; tuttavia, la memoization richiede spazio per memorizzare i valori intermedi, il che può diventare un problema se si devono calcolare numeri di Fibonacci molto grandi.

La programmazione dinamica è la soluzione più efficiente per il calcolo del numero di Fibonacci, perché utilizza la memoization per memorizzare i valori intermedi e ridurre la complessità computazionale; la complessità computazionale della programmazione dinamica è lineare, ovvero cresce linearmente con l'aumentare di n. In particolare, la complessità computazionale della programmazione dinamica per il calcolo del numero di Fibonacci è $O(n)$, il che significa che il tempo di esecuzione dell'algoritmo aumenta linearmente all'aumentare di n.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Divide et impera vs programmazione dinamica

La tecnica di Divide et Impera suddivide un problema in sottoproblemi indipendenti che possono essere risolti separatamente. Ogni sottoproblema viene poi risolto in modo ricorsivo e i risultati vengono combinati per ottenere la soluzione finale del problema originale. La tecnica di Divide et Impera è utile quando un problema può essere suddiviso in sottoproblemi indipendenti e quando è possibile combinarli facilmente.

D'altra parte, la tecnica di Programmazione Dinamica risolve un problema dividendo il problema in sottoproblemi sovrapposti che possono essere risolti una sola volta e i cui risultati possono essere riutilizzati in seguito. Ogni sottoproblema viene risolto in modo ricorsivo e i risultati vengono memorizzati in una tabella. La tecnica di Programmazione Dinamica è utile quando un problema può essere suddiviso in sottoproblemi sovrapposti e quando esiste una sovrapposizione significativa tra i sottoproblemi.

La tecnica di Divide et Impera può essere più efficiente della tecnica di Programmazione Dinamica quando ci sono pochi sottoproblemi da risolvere e quando i sottoproblemi possono essere risolti in modo efficiente. La tecnica di Programmazione Dinamica, d'altra parte, può essere più efficiente della tecnica di Divide et Impera quando ci sono molti sottoproblemi sovrapposti e quando i sottoproblemi non possono essere risolti in modo efficiente senza la memorizzazione dei risultati intermedi.

In generale, entrambe le tecniche di programmazione sono molto utili e sono ampiamente utilizzate nella risoluzione di problemi algoritmici complessi. La scelta tra le due tecniche dipende dalle caratteristiche specifiche del problema da risolvere e dalle prestazioni richieste dalla soluzione.

Ci sono molti esempi di problemi che possono essere risolti sia con la tecnica di Divide et Impera che con la tecnica di Programmazione Dinamica. Eccone alcuni esempi:

- Calcolo del numero di Fibonacci
- Calcolo della sequenza di Ackermann
- Ricerca del massimo sottovettore
- Ordinamento di un vettore
- […]

Analizziamo ad es. Fibonacci:

```
def fibonacci_divide_et_impera(n):
    if n <= 1:
        return n
    else:
        a = fibonacci_divide_et_impera(n-1)
        b = fibonacci_divide_et_impera(n-2)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        return a + b

def fibonacci_programmazione_dinamica(n):
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]
```

Entrambe le funzioni prendono come argomento il numero n di Fibonacci da calcolare. La funzione **fibonacci_divide_et_impera** utilizza la tecnica di Divide et Impera per calcolare il numero di Fibonacci in modo ricorsivo. Se n è minore o uguale a 1, la funzione restituisce n. Altrimenti, la funzione calcola il numero di Fibonacci come somma dei numeri di Fibonacci dei due numeri precedenti.

La funzione **fibonacci_programmazione_dinamica**, invece, utilizza la tecnica di Programmazione Dinamica per calcolare il numero di Fibonacci. La funzione utilizza una lista dp per memorizzare i risultati intermedi e calcola il numero di Fibonacci per ogni i da 2 a n. La soluzione finale viene restituita dall'elemento $dp[n]$.

Per esempio, per calcolare il 10° numero di Fibonacci, si può chiamare le due funzioni in questo modo:

```
fibonacci_divide_et_impera(10)  # Output: 55
fibonacci_programmazione_dinamica(10)  # Output: 55
```

Entrambe le funzioni restituiscono il valore 55, che è il 10° numero di Fibonacci. Tuttavia, la funzione fibonacci_programmazione_dinamica è più efficiente della funzione fibonacci_divide_et_impera, perché utilizza la programmazione dinamica per memorizzare i risultati intermedi e ridurre la complessità computazionale del calcolo dei numeri di Fibonacci.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione;
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione;
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley;
- Sedgewick: Algoritmi in C, Pearson, 2015;
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).